Supplementary Material for SGDAgent: Hierarchical Spatio-Temporal Scene Graph-Driven Agent for Structured Long-form Video Understanding

Anonymous Author(s)

This document is the supplementary material for the main paper. It is organized into five sections: (1) Construction cost and graph statistics, (2) Per-category performance breakdown, (3) Prompts and implementation details, (4) Algorithm pseudocode, and (5) Additional case studies.

1 Construction Cost and Graph Statistics

This section reports the offline construction cost and the resulting graph-database statistics for the archived 103-video LVBench[34] graph build used in our supplementary diagnostics.

Pipeline runtime. Offline construction is dominated by Stage-1 and Stage-2 visual extraction. With four parallel workers, the full LVBench[34] build completes in approximately 7.94 wall-clock hours. Stage-3 event graph construction on top of existing Stage-1/2 artifacts is inexpensive, completing in about 121 seconds for all 103 videos. This separation between costly visual extraction and cheap symbolic aggregation allows the graph database to be materialized once and reused across many downstream QA runs. Online agent inference adds only the cost of graph queries and selective frame inspection per question (see the efficiency analysis in Section 4.3 of the main paper).

Graph size. Table 6 summarizes the per-video statistics across the three graph levels. The sampling budget is fixed at 120 keyframes per video, so the spread primarily reflects scene complexity rather than raw video duration. Note that the segment count (122.6) and event-node count (120.9) are numerically close to the 120-frame budget by coincidence of the current windowing and merging thresholds—they do not imply a one-to-one correspondence between frames, segments, and events.

Entity deduplication. Frame graphs contain 3.65 entity mentions per sampled frame on average, yielding roughly 438 raw mentions per video over 120 frames. After greedy online alignment, the registry retains only 101.5 unique entities per video on average—approximately 76.8% of local mentions are absorbed into existing entries. The measured duplicate-entity rate after alignment is 0.0%, and the invalid-relation rate is 5.0% before aggregation.

Retrieval quality across levels. We measure Hit@1 (the fraction of questions whose top-1 retrieved window contains a frame supporting the gold answer) and Hit@k (k=5 for events, 6 for segments, 8 for frames; a hit if *any* of the top-k windows covers the evidence). Across LVBench[34], Stage-1 achieves 14.9 Hit@1 and 29.3 Hit@k; Stage-2 improves to 21.2 Hit@1 and 38.0 Hit@k; Stage-3 hybrid retrieval reaches 22.3 Hit@1 and 34.9 Hit@k with 100% non-empty retrieval and 73.1% non-empty frontiers. Stage-2 provides the best raw Hit@k, while Stage-3 improves top-ranked event localization and frontier quality with full retrieval coverage. This retrieval-level trend complements the end-to-end ablation in Table 3 of the main paper: Stage-2 drives accuracy gains through richer candidate coverage, whereas Stage-3 primarily improves search efficiency by narrowing the candidate set in fewer steps.

Table 6: Per-video graph statistics on the 103-video LVBench[34] build. All values are averages unless noted.

Level	Metric	Value
Stage-1 (Frame)	Unique entities	101.5
	Frame-level relations	270.8
	Empty frames	0%
Stage-2 (Segment)	Segments	122.6
	Segment-entity mentions	394.0
	Temporal relations	316.4
Stage-3 (Event)	Event nodes	120.9
	Event edges	307.8
	Avg. uncertainty	0.199

Table 7: Per-category end-to-end accuracy (%) on LVBench[34] under matched settings. n is the number of questions carrying each official LVBench capability tag.

Category	n	Caption-only	SGDAgent
Entity Recognition	677	55.2	71.3
Event Understanding	647	54.1	71.9
Key Info. Retrieval	291	65.6	83.2
Temporal Grounding	220	54.1	76.3
Reasoning	201	48.3	60.5
Summarization	58	43.1	68.6

2 Per-Category Performance Breakdown

LVBench[34] annotates each question with one or more official capability tags. The most frequent are entity recognition (677 / 43.7%), event understanding (647 / 41.8%), key information retrieval (291 / 18.8%), temporal grounding (220 / 14.2%), reasoning (201 / 13.0%), and summarization (58 / 3.7%).

Table 7 reports per-category accuracy under matched settings. The gains are broad rather than isolated to a single question family. The largest improvements appear on temporal grounding (+22.2 points) and summarization (+25.5 points, albeit on a small 58-question slice), while event understanding (+17.8), key information retrieval (+17.6), and entity recognition (+16.1) also improve substantially. This trend is consistent with the main-paper claim that SGDAgent is strongest when the question first requires narrowing a long video to temporally plausible regions before local visual verification.

Table 8: Prompt modules used by SGDAgent.

Module	Stage	Role
Stage-1 frame graph prompt	offline	Convert one keyframe into strict JSON scene-graph output with typed entities and relation triples.
Orchestrator system prompt	online	Enforce the stepwise tool loop and finish-only termination (finish corresponds to ANSWER in the main paper).
Tool instruction prompt	online	Define tool semantics and the localization-versus-verification contract.
Runtime policy injection	online	Add question-specific verification constraints, including repeated targeted FI and time-valid terminal evidence.
Frame-inspect prompt	online	Ask the VLM to verify raw-frame evidence inside a provided time range.

The smallest, though still positive, gain appears on reasoning questions (+12.2). Many of these questions ultimately hinge on fine-grained visual evidence such as color, counting, appearance, or transient state, where the graph serves primarily as a localization prior and terminal confirmation still depends on FI. This is also why the control strategies in the main paper emphasize routing targeted questions toward visual verification rather than allowing graph text alone to finalize the answer.

3 Prompts and Implementation Details

This section provides the prompt templates and key implementation details needed for reproducibility. Table 8 summarizes all prompt modules.

3.1 Scene Graph Construction Prompt

Frame-level graphs are extracted offline with gpt-4.1 (temperature 0, max 800 tokens). Keyframes are sampled every 5 seconds from 2 FPS decoded frames, keeping at most 120 per video. The Stage-1 prompt is:

```
System:
You produce strict JSON only.

User:
Extract a compact frame-level scene graph from this image.
Return strict JSON with schema:
{"entities":[{"local_id":"e1","name":"person","type":"person",
"attributes":["standing"],"desc_text":"short description",
"confidence":0.0}],
"relations":[{"s":"e1","p":"holding","o":"e2","confidence":0.0}]}.
Rules: keep 1-8 salient entities, 0-12 relations, relation
predicate in snake_case.
```

Post-processing applies deterministic normalization: entity names are lower-cased; entity types are restricted to a fixed vocabulary (person, object, scene, animal, vehicle, food, text, location, building, clothing, instrument, plant) with out-of-vocabulary types mapped to object; a blacklist removes meta-level terms (video, camera, subtitle, background); relation predicates are lower-cased and filtered by a whitelist covering spatial, interaction, and state/action predicates; confidence values are clipped to $[0, 1]$. If the VLM returns no usable entities, a caption fallback extracts up to four coarse entities from the nearest caption segment (confidence 0.2).

3.2 Entity Alignment

The video-level entity registry uses greedy online matching. For a new frame entity e and registry entity $\tilde{e}$, the alignment score is

$$R_{\text{align}}(e, \tilde{e}) = 0.7\, R_{\text{name}}(e, \tilde{e}) + 0.2\, R_{\text{desc}}(e, \tilde{e}) + 0.1\, R_{\text{type}}(e, \tilde{e}), \quad (1)$$

where R_{name} and R_{desc} are SequenceMatcher ratios on the name and description fields, and $R_{\text{type}} = 1$ if types match exactly. The merge threshold is $\tau = 0.78$. On merge, last_seen is updated, confidence takes the max, new aliases are appended, and attributes are merged by set union.

3.3 Segment and Event Construction Hyperparameters

Stage-2 segments. Overlapping sliding windows with window size 30s, overlap 5s, stride 25s, and a cap of 64 relations per segment. Frame-level triples sharing the same (s, p, o) signature merge into temporal relations spanning the earliest to latest supporting frame.

Stage-3 events. Temporally adjacent segments cluster into events when the temporal gap $\leq$ 12s *and* at least one semantic condition holds: relation Jaccard $\geq$ 0.18, or shared entity count $\geq$ 1 with predicate Jaccard $\geq$ 0.22, or repeated entity co-occurrence with any shared predicate (see MERGEABLE in Algorithm 3). Within each cluster, consecutive segments are grouped into temporal units; a new unit starts when the inter-segment gap exceeds $\delta_{\text{split}} = 8$s, the cumulative span exceeds a hard cap of 75s, or exceeds a soft cap of 45s with continuity below 0.5. Group events form from $\geq$2 units via `parent-child` edges (confidence 0.95). Pairwise edges (`before`, `overlaps`, `entity-carryover`, `same-thread`) are inferred from temporal ordering and entity sharing; `same-thread` edges additionally require a gap $\leq \delta_{\text{thread}} = 90$s.

3.4 Agent Prompts

```
You are a helpful assistant who answers multi-step questions by
sequentially invoking functions. Follow the THINK -> ACT -> OBSERVE
loop:
  * THOUGHT Reason step-by-step about which function to call next.
  * ACTION   Call exactly one function that moves you closer to the
             final answer.
  * OBSERVATION Summarize the function's output.
You MUST plan extensively before each function call, and reflect
extensively on the outcomes of the previous function calls.
Only pass arguments that come verbatim from the user or from earlier
function outputs---never invent them.
Continue the loop until the user's query is fully resolved, then
call the `finish` function with the final answer.
Do not answer directly in assistant text when you are ready to
finish.
```

The implementation-level finish tool corresponds to the terminal ANSWER action in Algorithm 1 of the main paper.

Tool-instruction prompt. The tool instructions define three roles. First, they specify the retrieval stack: graph retrieval for structured

proposal, caption backoff for weak hits, and FI for visual verification. Second, they enforce the contract "graph for localization, vision for confirmation" so that precise answers are not finalized from graph text alone. Third, they define how to interpret controller_signals from graph retrieval outputs: localize_only, weak_match, recomm end_frame_inspect, no_temporal_hit, and descend_recommended.

```
System:
You are a helpful assistant to answer questions.

User:
Carefully watch the video frames. Pay attention to the cause and
sequence of events, the detail and movement of objects and the
action and pose of persons.
Based on your observations, answer the question using the visual
evidence. If no relevant content is found within the given time
range, return: `Error: Cannot find corresponding result in the
given time range.`.
Question: <question>
```

The implementation samples up to 50 frames uniformly over the provided time ranges before sending them to gpt-4.1-mini.

3.5 Control Strategy Details

Strategy R (query-type routing). A QuestionPolicy is built from question-type hints, explicit time references, and lexical cues. It determines whether the question requires targeted visual check (requires_targeted_fi), repeated confirmation (requires_repeated_ targeted_fi), or time-valid terminal evidence (requires_time_valid_ terminal). The targeted label is one of: count, color, on-screen text, identity, appearance, or time-localized event.

Strategy G (confidence gating). Graph outputs are rejected as unreliable when they are fallback-only, have top lexical score below 0.15, contain only generic entities with no relations, miss an explicit target time span, or have insufficient top-1 margin (threshold 0.12). The graph-quality score is

$$Q_{\text{graph}} = S_{\text{top}} + 2S_{\text{lex,top}} + |\mathcal{E}_{\text{match}}| + 1.5|\mathcal{R}_{\text{match}}| + \gamma_{\text{time}}, \quad (2)$$

where $S_{\text{top}} = S(w_1 \mid q)$ is the composite retrieval score of the top-ranked candidate (Eq. 5 in the main paper), $S_{\text{lex,top}}$ is its lexical component, and $\gamma_{\text{time}} = 1.5$ if time-valid, otherwise the overlap ratio. The finish gate requires at least one narrow targeted `FI` record for targeted questions; for repeated confirmation, two narrow `FI` calls must overlap by ≥0.5 and agree by exact signature or text-token Jaccard ≥0.55.

Strategy T (terminal convergence). A sliding stagnation window of 3 steps monitors progress. If no improvement in graph quality, clip overlap, or qualified FI evidence, the controller activates convergence mode. At most 3 finish rejections are allowed before the gate relaxes. On the final iteration, finish is forced; if it still fails, a hard choice-only extraction ensures benchmark robustness.

FTT definition. A trajectory counts as failed-to-terminate (FTT) if no valid option from {A, B, C, D} can be recovered at execution end, whether due to runtime failure, budget exhaustion without a terminal answer, or non-convergence after the forced-answer safeguard. The four metrics in the main paper satisfy: answer rate = 1 − FTT rate, and end-to-end accuracy counts all FTT cases as incorrect.

Algorithm 1 Stage-1 Entity Alignment into Video-Level Registry

Require: frame parse $(V_t^f, \mathcal{R}_t^f)$ at time t, caption context c_t, registry E^{video}, threshold τ, predicate whitelist $\mathcal{P}_{\text{allow}}$
Ensure: aligned frame graph $(\bar{V}_t^f, \bar{\mathcal{R}}_t^f)$, updated registry E^{video}

if $V_t^f = \emptyset$ **and** caption fallback is enabled **and** $c_t \neq \emptyset$ **then**
  $(V_t^f, \mathcal{R}_t^f) \leftarrow \textsc{FallbackParse}(c_t)$
end if
$M \leftarrow \emptyset$, $\bar{V}_t^f \leftarrow [\,]$, $\bar{\mathcal{R}}_t^f \leftarrow [\,]$
for all $e \in V_t^f$ **do**
  $j^\star \leftarrow -1$, $r^\star \leftarrow -\infty$
  for all $\tilde{e}_j \in E^{\text{video}}$ **do**
    $r \leftarrow 0.7R_{\text{name}}(e, \tilde{e}_j) + 0.2R_{\text{desc}}(e, \tilde{e}_j) + 0.1R_{\text{type}}(e, \tilde{e}_j)$
    if $r > r^\star$ **then**
      $r^\star \leftarrow r$, $j^\star \leftarrow j$
    end if
  end for
  if $j^\star \geq 0$ **and** $r^\star \geq \tau$ **then**
    $\tilde{e} \leftarrow E^{\text{video}}[j^\star]$
    $\tilde{e}$.last_seen $\leftarrow t$
    $\tilde{e}$.confidence $\leftarrow \max(\tilde{e}$.confidence, e.confidence$)$
    if e.name $\neq \tilde{e}$.canonical_name **and** e.name $\notin \tilde{e}$.aliases **then**
      append e.name to $\tilde{e}$.aliases
    end if
    $\tilde{e}$.attributes $\leftarrow \tilde{e}$.attributes $\cup$ e.attributes
    $g(e) \leftarrow \tilde{e}$.entity_id
  else
    create $\tilde{e}_{\text{new}}$ with new ID, canonical name, type, attributes, confidence, first/last seen, and empty evidence
    append $\tilde{e}_{\text{new}}$ to E^{video}
    $g(e) \leftarrow \tilde{e}_{\text{new}}$.entity_id
  end if
  $M[e$.local_id$] \leftarrow g(e)$
  append $(g(e), e$.name, e.type, e.confidence$)$ to $\bar{V}_t^f$
end for
$\mathcal{D} \leftarrow \emptyset$
for all $r = (u, p, v) \in \mathcal{R}_t^f$ **do**
  if $p \notin \mathcal{P}_{\text{allow}}$ **then**
    continue
  end if
  $s \leftarrow M[u]$, $o \leftarrow M[v]$
  if $s = \emptyset$ **or** $o = \emptyset$ **or** $(s, p, o) \in \mathcal{D}$ **then**
    continue
  end if
  append $(s, p, o, r$.confidence$)$ to $\bar{\mathcal{R}}_t^f$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, p, o)\}$
end for
for all $\tilde{e} \in \bar{V}_t^f$ **do**
  append current frame ID to the evidence list of $\tilde{e}$.entity_id in E^{video}
end for
return $(\bar{V}_t^f, \bar{\mathcal{R}}_t^f), E^{\text{video}}$

4 Algorithm Pseudocode

This section provides four pseudocode blocks that complement Algorithm 1 in the main paper: Stage-1 entity alignment (Algorithm 1), Stage-2 segment graph aggregation (Algorithm 2), Stage-3 global event graph construction (Algorithm 3), and Strategy G confidence gating (Algorithm 4).

5 Additional Case Studies

This section presents four representative trajectories—two successful and two failed—that complement the main-paper case study

Algorithm 2 Stage-2 Segment Graph Aggregation

Require: frame graph G^f, window size $w = 30$, overlap $o = 5$, relation cap $K_{\text{rel}} = 64$
Ensure: segment graph G^s

$\mathcal{W} \leftarrow \textsc{SlidingWindows}([0, T_V], w, o)$
$G^s \leftarrow \varnothing$
for all $[a, b] \in \mathcal{W}$ **do**
  $\mathcal{F}_{ab} \leftarrow \{f \in G^f \mid t_f \in [a, b)\}$; include $t_f = b$ only for the last window
  if $\mathcal{F}_{ab} = \varnothing$ **then**
    continue
  end if
  $A \leftarrow \varnothing, R \leftarrow \varnothing, F_{\text{sup}} \leftarrow [\,]$
  for all $f \in \mathcal{F}_{ab}$ **do**
    append f.frame_id to F_{sup}
    for all $e \in V_f$ **do**
      if e.entity_id $\notin A$ **then**
        initialize $A[e.\text{entity_id}]$ with canonical name, type, confidence, and frame count 1
      else
        update confidence by max and increment frame count
      end if
    end for
    for all $r = (s, p, o) \in \mathcal{R}_f$ **do**
      if $(s, p, o) \notin R$ **then**
        initialize temporal relation with $t_{\text{start}} = t_f$, $t_{\text{end}} = t_f$, count 1, confidence, and support frame IDs
      else
        $R[(s, p, o)].t_{\text{start}} \leftarrow \min(R[(s, p, o)].t_{\text{start}}, t_f)$
        $R[(s, p, o)].t_{\text{end}} \leftarrow \max(R[(s, p, o)].t_{\text{end}}, t_f)$
        update confidence by max, append frame ID if new, increment count
      end if
    end for
  end for
  active_entities $\leftarrow \textsc{Sort}(A,$ by frame count, confidence$)$
  temporal_relations $\leftarrow \textsc{TopK}(\textsc{Sort}(R,$ by count, confidence$), K_{\text{rel}})$
  $\sigma \leftarrow \textsc{Summary}$(active_entities, temporal_relations)
  $n \leftarrow \textsc{BuildSegmentNode}(a, b,$ active_entities, temporal_relations, $F_{\text{sup}}, \sigma)$
  append n to G^s
end for
for $i = 2$ to $|G^s|$ **do**
  $\mathcal{E}_{i-1} \leftarrow$ entity IDs in $G^s[i-1]$, $\mathcal{E}_i \leftarrow$ entity IDs in $G^s[i]$
  $\mathcal{E}_\cap \leftarrow \mathcal{E}_{i-1} \cap \mathcal{E}_i$
  if $|\mathcal{E}_\cap| > 0$ **then**
    attach `overlap_link`$(G^s[i], G^s[i-1], \mathcal{E}_\cap[1{:}16], |\mathcal{E}_\cap|)$
  end if
end for
return G^s

(Figure 3). Figures 4–6 illustrate three of the four cases; Failure Pattern 1 is described in text only. All traces use the matched-setting configuration with tools limited to `GR`, `SR`, `FR`, and `FI`.

5.1 Successful Patterns

Pattern 1: graph-guided recovery after an incorrect local probe (Figure 4). The question "How many white lights were on at 45:08?" begins with `FI`, which immediately returns an incorrect count of 34. The controller then switches to `GR` and `SR`: event `evt_0053` (2675–2730s) anchors to `stage` and `lights` (`ent_000023`), and segment `seg_0108` (2675–2705s) maintains the same lighting configuration. The graph cannot count individual bulbs, but it repairs the temporal search branch; a narrow `FI` call on 00:45:08–00:45:09 then yields the correct answer of 24. This case demonstrates that the graph's value lies in structural re-localization, not direct symbolic counting.

Pattern 2: relation-aware anchoring around stable scene structure (Figure 5). For "Where does the girl take the vampire after the exam?", the trace is short: `GR`→`SR`→`FI`. Event `evt_0071` (2950–3030s) captures the right participants (`vampire`, `girl`) with

Algorithm 3 Stage-3 Global Event Graph Construction

Require: segment graph G^s, $\delta_{\text{gap}} = 12$, $\tau_{\text{rel}} = 0.18$, $\tau_{\text{pred}} = 0.22$, $\delta_{\text{split}} = 8$, $\delta_{\text{thread}} = 90$
Ensure: event graph G^g

function $\textsc{Mergeable}(C, s)$
  $\Delta t \leftarrow t_{\text{start}}(s) - t_{\text{end}}(C)$
  $m \leftarrow |E(C) \cap E(s)|$
  $J_{\text{rel}} \leftarrow \textsc{Jaccard}(R(C), R(s))$
  $J_{\text{pred}} \leftarrow \textsc{Jaccard}(P(C), P(s))$
  repeat $\leftarrow (m \geq 1) \wedge (J_{\text{pred}} > 0)$
  return $(\Delta t \leq \delta_{\text{gap}}) \wedge \big(J_{\text{rel}} \geq \tau_{\text{rel}} \vee (m \geq 1 \wedge J_{\text{pred}} \geq \tau_{\text{pred}}) \vee \text{repeat}\big)$
end function
sort segments in G^s by t_{start}
$C \leftarrow [\,]$
for all $s \in G^s$ **do**
  if $C = \varnothing$ **or** $\neg\textsc{Mergeable}(C[-1], s)$ **then**
    append new cluster $[s]$ to C
  else
    append s to $C[-1]$
  end if
end for
$V^g \leftarrow \varnothing, \mathcal{E}^g \leftarrow \varnothing$
for all $C \in C$ **do**
  $\mathcal{U} \leftarrow [\,]$, start first temporal unit u
  for all $s \in C$ in temporal order **do**
    if $u = \varnothing$ **then**
      append s to u
    else if $\textsc{Gap}(u, s) > \delta_{\text{split}}$ **or** $\textsc{Span}(u \cup \{s\}) > 75$ **or** $|u| + 1 > 2$ **then**
      finalize u and start a new temporal unit with s
    else if $\textsc{Span}(u \cup \{s\}) > 45$ **and** $\textsc{Continuity}(u, s) < 0.5$ **then**
      finalize u and start a new temporal unit with s
    else
      append s to u
    end if
  end for
  finalize the last unit and append all units to $\mathcal{U}$
  for all $u \in \mathcal{U}$ **do**
    $n \leftarrow \textsc{BuildUnitEvent}(u, 8, 10)$
    attach node confidence and uncertainty to n
    append unit event n to V^g
  end for
  if $|\mathcal{U}| \geq 2$ **and** group-event emission is enabled **then**
    $g \leftarrow \textsc{BuildGroupEvent}(C, 10, 12)$
    attach confidence and uncertainty to g
    append group event g to V^g
    for all $u \in \mathcal{U}$ **do**
      add `parent-child` edge (g, u) with confidence 0.95 to $\mathcal{E}^g$
    end for
  end if
end for
for all ordered pair (n_i, n_j) in V^g, $i \neq j$ **do**
  if $t_i^e < t_j^s$ **then**
    add `before` edge
  end if
  if $[t_i^s, t_i^e]$ overlaps $[t_j^s, t_j^e]$ **then**
    add `overlaps` edge
  end if
  if $|P_i \cap P_j| > 0$ **then**
    add `entity-carryover` edge
    if $\textsc{Gap}(n_i, n_j) \leq \delta_{\text{thread}}$ **then**
      add `same-thread` edge
    end if
  end if
end for
return $G^g = (V^g, \mathcal{E}^g)$

`sitting_on` and `next_to` relations. Segment `seg_0116` (2875–2905s) surfaces `person sitting_on bench` and `bench under tree`, keeping the search inside the correct park-like scene cluster. Although the graph canonicalizes characters as generic `person` nodes, the relational backbone around `bench` and `tree` is sufficient

Algorithm 4 Strategy G: Weak-Signal Rejection and Finish Gate

Require: retrieval result set $\mathcal{W}$, question policy π from Strategy R (encodes `targeted`, `repeat`, `time`, `fine` flags), agent state H

function WeakSignal($\mathcal{W}, \pi$)
  $w_1 \leftarrow \text{Top}(\mathcal{W})$
  no_hit $\leftarrow$ FallbackUsed($\mathcal{W}$) $\wedge$ $S_{\text{lex}}(w_1) \leq 0 \wedge \neg$EmbeddingStrongHit($w_1$)
  low_lex $\leftarrow S_{\text{lex}}(w_1) < 0.5$
  rel_empty $\leftarrow |R_{\text{match}}(w_1)| = 0$
  weak $\leftarrow$ no_hit $\vee$ (low_lex $\wedge$ rel_empty $\wedge |E_{\text{match}}(w_1)| = 0) \vee S(w_1) < 0.5$
  coarse $\leftarrow$ Width(w_1) ≥ 25
  localize_only $\leftarrow$ weak $\vee\ \pi$.fine $\vee$ (coarse $\wedge$ (π.fine $\vee$ no_hit $\vee$ (low_lex $\wedge$ rel_empty)))
  $(v_{\text{time}}, \rho_{\text{time}}) \leftarrow$ TimeValidity(w_1, π)
  $Q_{\text{graph}} \leftarrow S(w_1) + 2S_{\text{lex}}(w_1) + |E_{\text{match}}(w_1)| + 1.5|R_{\text{match}}(w_1)| +$ (1.5 **if** v_{time} **else** ρ_{time})
  $\Gamma \leftarrow \emptyset$
  if FallbackUsed($\mathcal{W}$) **then**
    $\Gamma \leftarrow \Gamma \cup \{$`fallback_only`$\}$
  end if
  if $S_{\text{lex}}(w_1) < 0.15$ **then**
    $\Gamma \leftarrow \Gamma \cup \{$`lexical_low`$\}$
  end if
  if AllGeneric($E_{\text{match}}(w_1)$) $\wedge\ |R_{\text{match}}(w_1)| = 0$ **then**
    $\Gamma \leftarrow \Gamma \cup \{$`generic_entity_only`$\}$
  end if
  if TopMargin($\mathcal{W}$) < 0.12 **then**
    $\Gamma \leftarrow \Gamma \cup \{$`top_margin_low`$\}$
  end if
  if π.time **and** $\neg v_{\text{time}}$ **then**
    $\Gamma \leftarrow \Gamma \cup \{$`target_span_miss`$\}$
  end if
  return $(\Gamma \neq \emptyset), Q_{\text{graph}}$, localize_only
end function
function GatingOK(H, π)
  $\mathcal{F} \leftarrow \{r \in H.\text{FI} \mid (\neg\pi.\text{time} \vee r.\text{time_valid}) \wedge (\neg\pi.\text{targeted} \vee (r.\text{targeted} \wedge r.\text{narrow}))\}$
  if π.targeted **and** $\mathcal{F} = \emptyset$ **then**
    return false
  end if
  if π.repeat **then**
    if $|\mathcal{F}| < 2$ **then**
      return false
    end if
    $(r_{-1}, r_0) \leftarrow$ last two records in $\mathcal{F}$
    if OverlapRatio(r_{-1}, r_0) < 0.5 **then**
      return false
    end if
    if $\neg$Consistent(r_{-1}, r_0) **then**
      return false
    end if
  end if
  if π.time **and** $\mathcal{F} = \emptyset$ **and** $\neg$HasTimeValidClip(H) **then**
    return false
  end if
  return true
end function

for localization. The final `FI` call only needs to distinguish among location categories, not rediscover the event from scratch.

5.2 Failure Patterns

Pattern 1: event windows too coarse for transient visual attributes. The opening-light question for a Vancouver gala performance fails despite broadly on-topic event retrieval. `GR` returns `evt_0012` (475–655s), a seven-segment event involving `person`, `ice_rink`, and `audience`, but with no relation pinning down the exact opening instant. The `FI` call on 00:09:35–00:10:05 sees the dominant white illumination of the broader performance and misses the correct opening color. This is a granularity mismatch: the attribute changes on a timescale shorter than the current event support span.

Pattern 2: correct process localization fails without count/state relations (Figure 6). The Taiyaki-capacity question shows the representational boundary. `GR` retrieves `evt_0038` (1750–1780s) near `batter`, `waffle_iron`, and `machine`; `SR` surfaces `taiyaki_mold`, `taiyaki`, and `taiyaki_maker`. The graph correctly localizes the cooking phase, but neither Stage-2 nor Stage-3 represents mold-slot occupancy as an explicit relation. The final `FI` call overcounts visible shapes and answers six instead of four.

5.3 Takeaways

The main advantage of SGDAgent across these cases is that the hierarchical graph regularizes search around reusable structural units: stable participants, recurring event motifs, and cross-segment relations. The main shortfalls are also structural: generic entities are merged too aggressively for instance-level disambiguation, transient attributes can change faster than current event windows, and counting problems still depend on `FI` because the graph does not encode numerical state. These observations suggest that the most impactful future improvements are richer instance identity, finer temporal slicing, and explicit count/state relations.

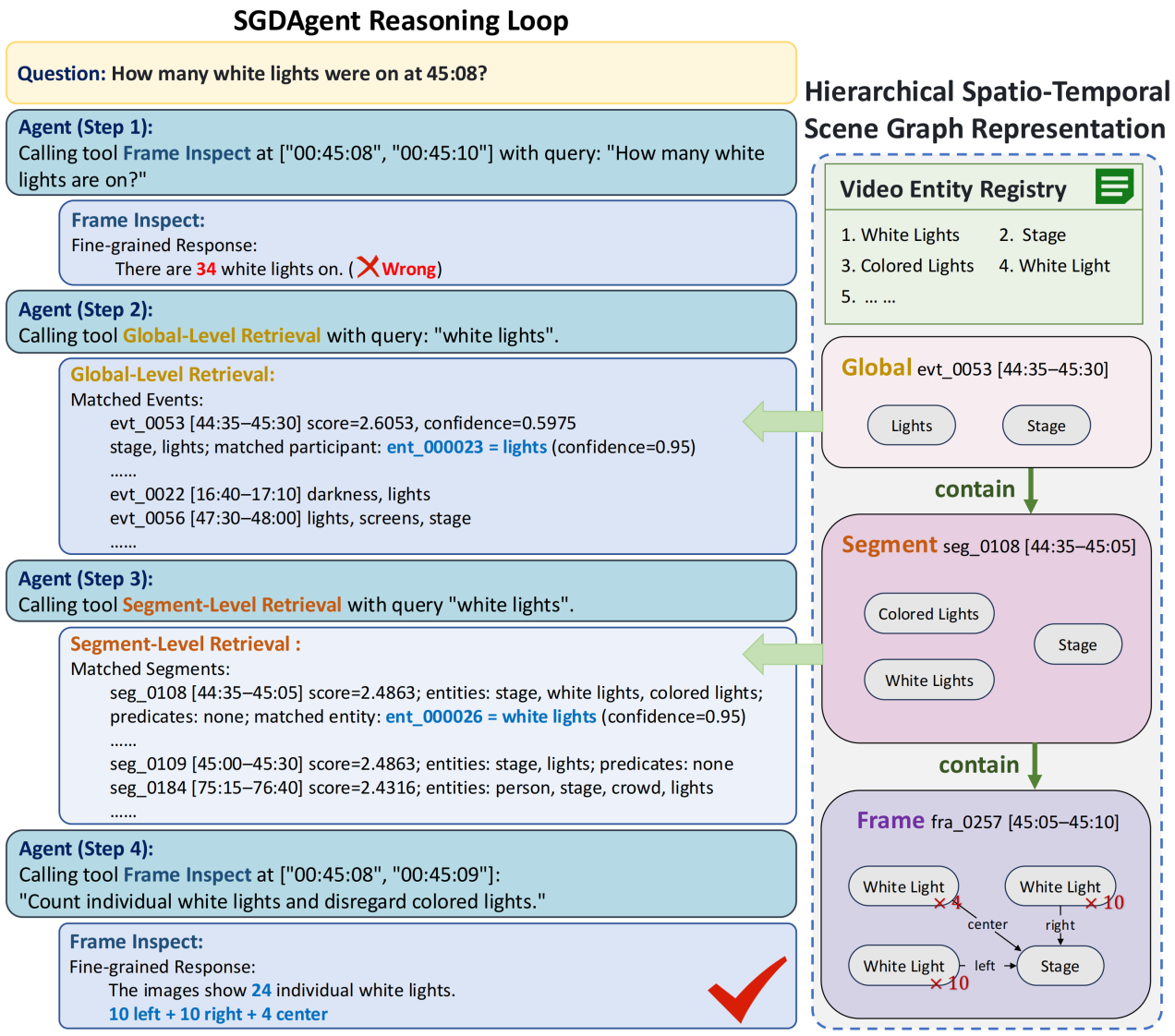


Figure 4: Case study from LVBench[34], (qid: 2256). SGDAgent reasoning trajectory on an exact counting example: graph-guided retrieval repairs an initially incorrect local probe and guides targeted visual verification to the correct answer.

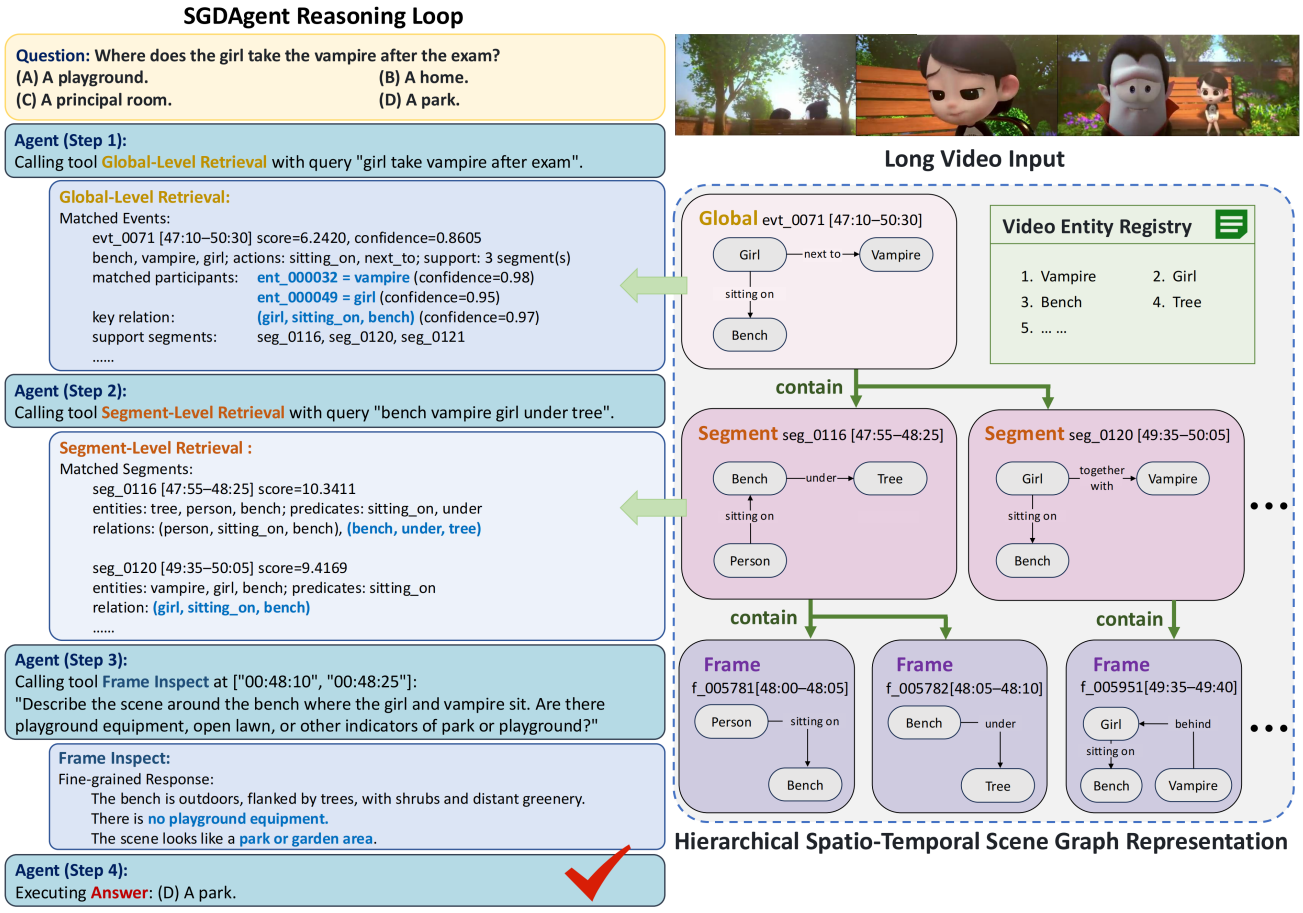


Figure 5: Case study from LVBench[34], (qid: 412). SGDAgent reasoning trajectory on a location QA example: graph queries progressively localize the answer around stable entity-relation structure before a single visual verification step commits the final answer.

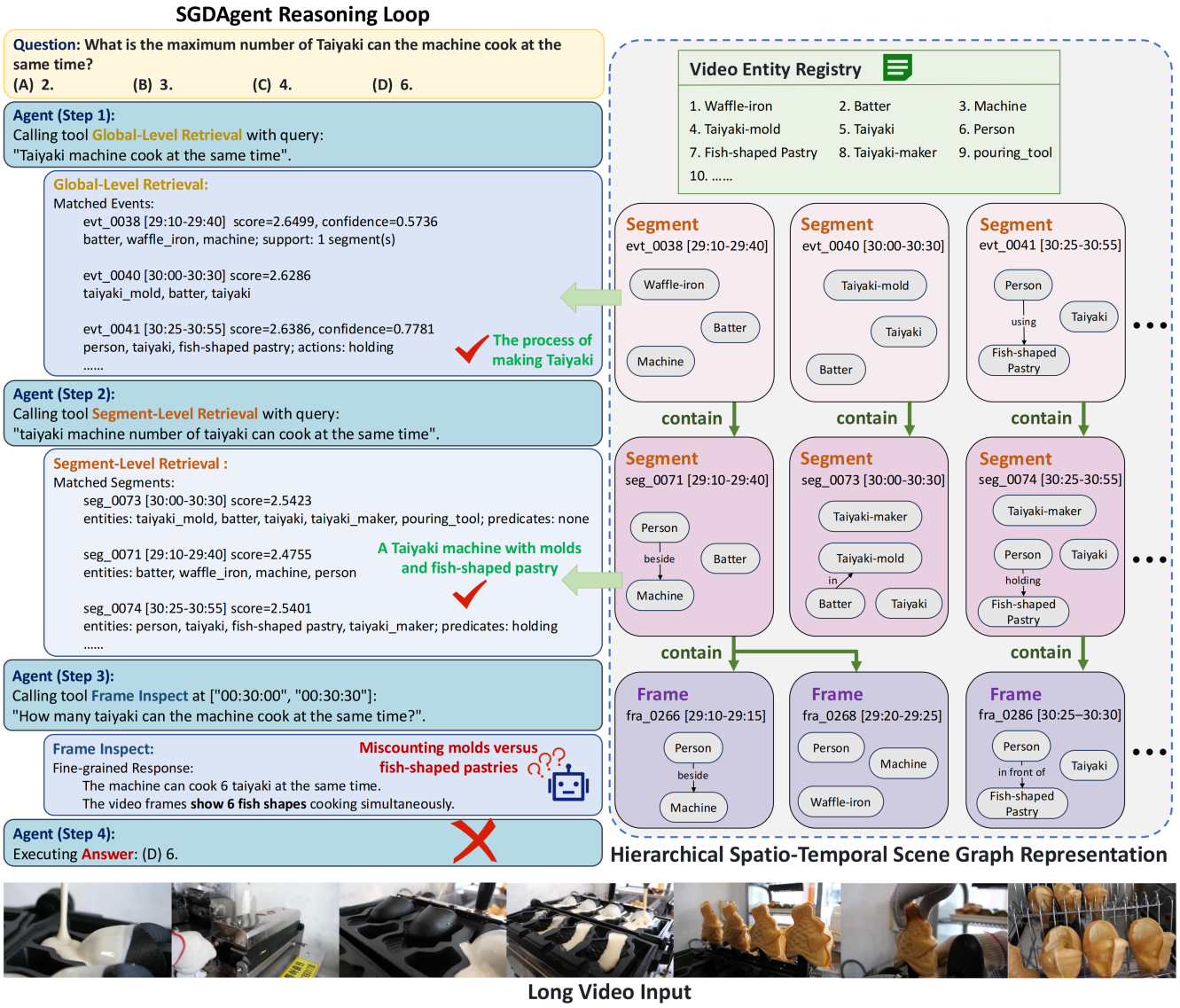


Figure 6: Case study from LVBench[34], (qid: 3322). Failure case on numerical-state reasoning: the graph correctly localizes the cooking process, but the final visual inspection still overestimates the Taiyaki machine capacity.